

Data Spec

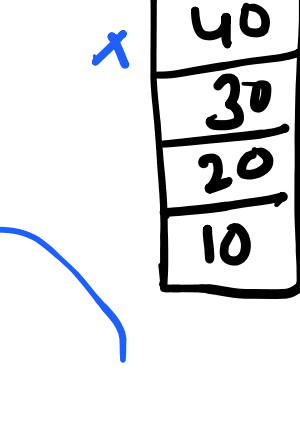
Stack/Queue

① Stack

LIFO → Application

101 201 301 401 501

101 201 301 401 501



push → add
peek → get → Top
pop → remove → Top

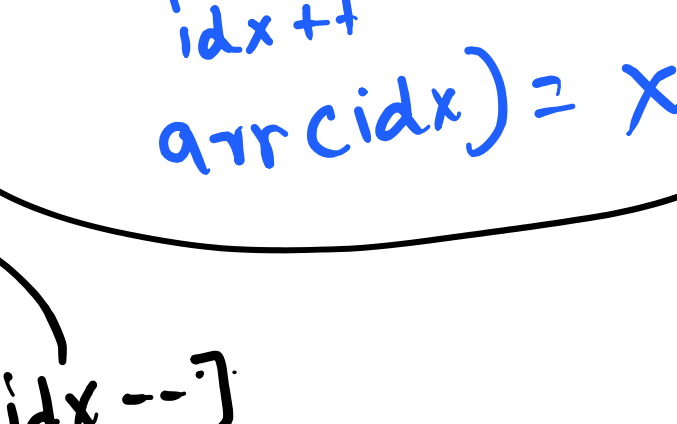
```
public class Stack {  
    private int[] arr;  
    private int idx; → null  
    public Stack() {  
        // TODO Auto-generated constructor stub  
    }  
    public Stack(int n) {  
        // TODO Auto-generated constructor stub  
    }  
}
```

```
public class StackClient {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        Stack st = new Stack();  
        Stack st1 = new Stack(7);  
    }  
}
```

idx = arr.length

Size = idx + 1

```
public void push(int x) {  
    // ...  
}
```



st.push(10) ←
st.push(20)
st.push(30)
st.push(40)
st.push(50)

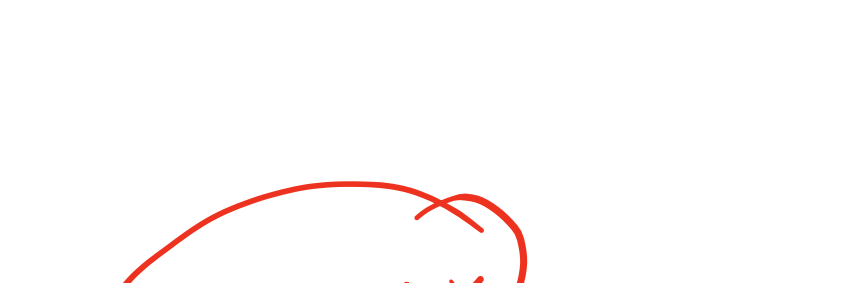
arr[idx] = x

idx++
arr[idx] = x

x = arr[idx--]

```
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
    Stack st = new Stack();  
    Stack st1 = new Stack(7);  
    st.push(10);  
    st.push(20);  
    st.push(30);  
    st.push(40);  
    st.push(50);  
    System.out.println(st.peek());  
    System.out.println(st.pop());  
    st.push(60);  
    st.push(70);  
    st.Display();  
}
```

idx = 1 0 1 2 3 4 5 6

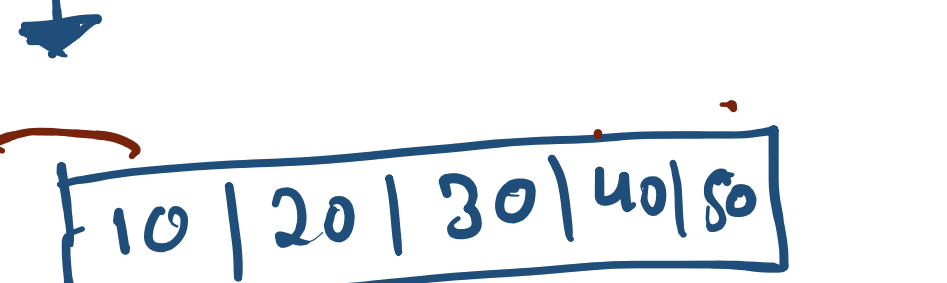
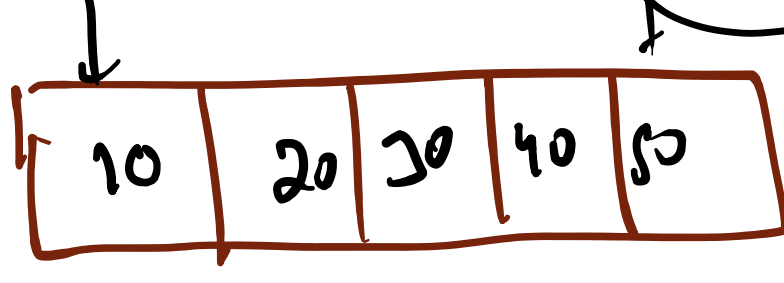


2015 → 5015

Queue → interface

FIFO →

LIFO

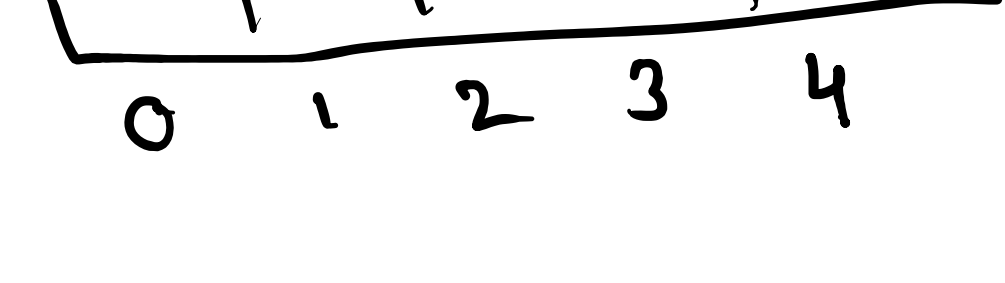


Enqueue → add
delete → dequeue
get front → view

front →

Size = 3

2015



```
public void Dequeue(int x) throws Exception {  
    if (isEmpty()) {  
        throw new Exception("Bklol Queue Empty hai");  
    }  
}
```

x = arr[front]
front++
size--

```
public void Enqueue(int x) throws Exception {  
    if (isfull()) {  
        throw new Exception("Bklol Queue full hai");  
    }  
    arr[size++] = x;  
}
```

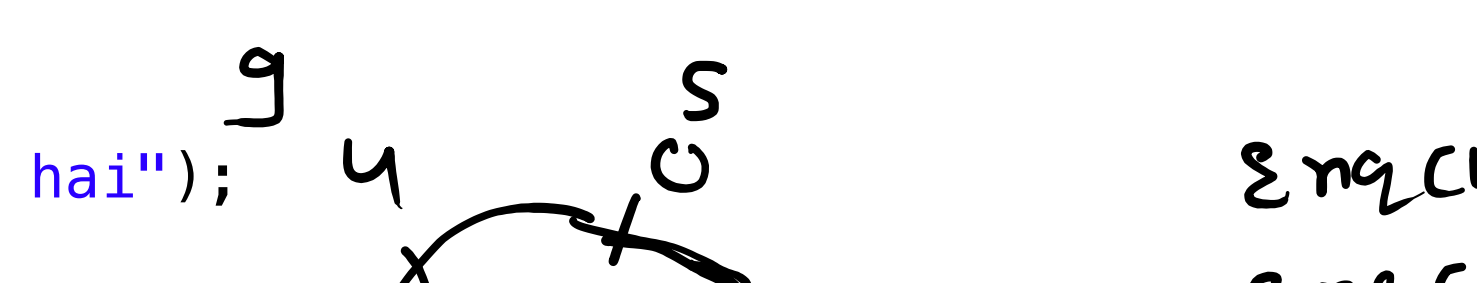
Size = 4
front = 0 + 2

```
public void Dequeue() throws Exception {  
    if (isEmpty()) {  
        throw new Exception("Bklol Queue Empty hai");  
    }  
    int x = arr[front];  
    front++;  
    size--;  
    return x;  
}
```

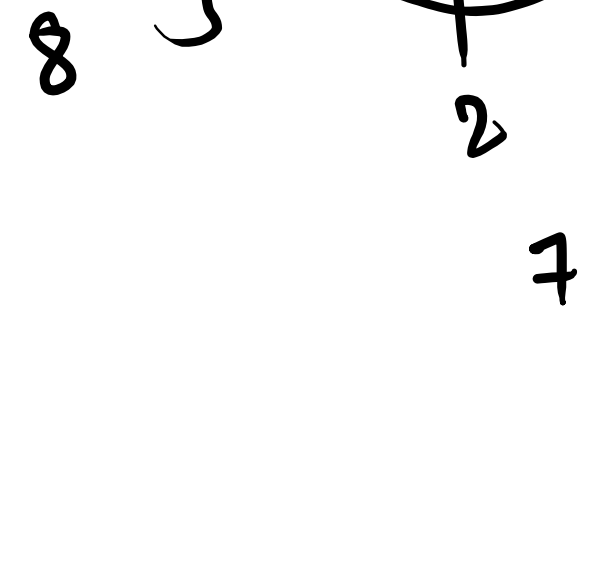
2015
2020
2030
2040
2050

```
public void Enqueue(int x) throws Exception {  
    if (isfull()) {  
        throw new Exception("Bklol Queue full hai");  
    }  
    int idx = front + size;  
    arr[idx] = x;  
    size++;  
}
```

F = 2 * 2
S = 2 * 2



```
public int Dequeue() throws Exception {  
    if (isEmpty()) {  
        throw new Exception("Bklol Queue Empty hai");  
    }  
    int x = arr[front];  
    front++;  
    size--;  
    return x;  
}
```



2015
2020
2030
2040
2050

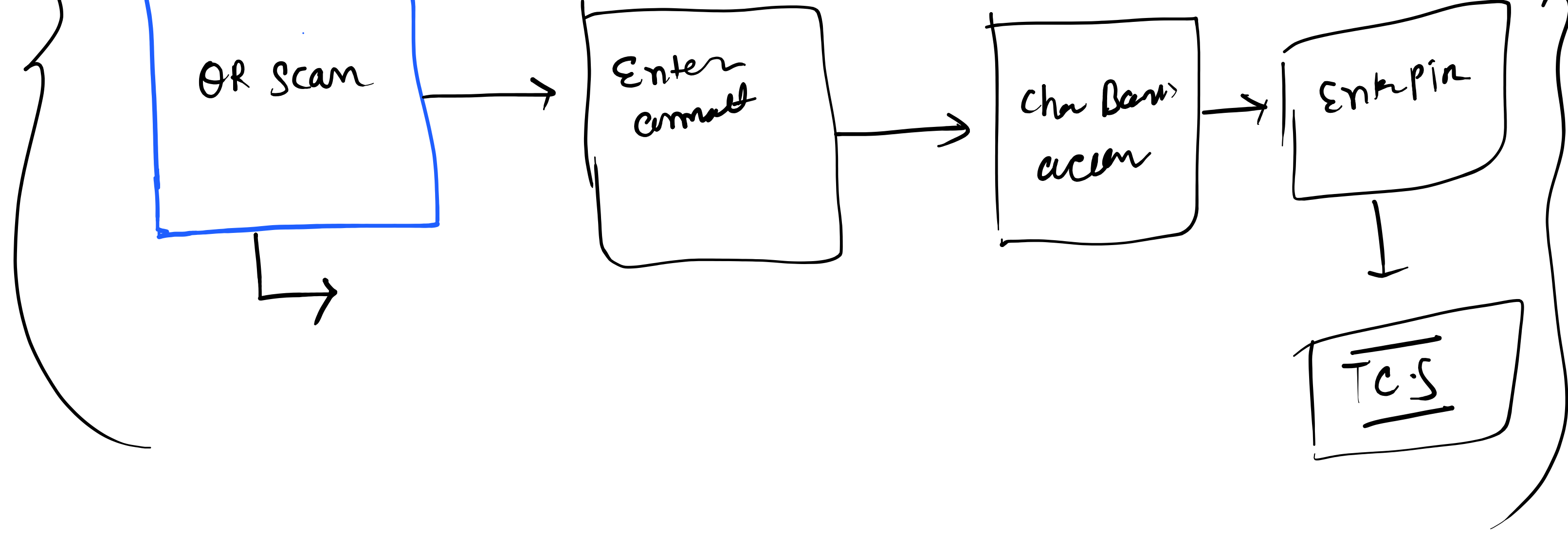
30 40 50

30 40 50 60

front = 0
idx = (front + i) % arr.length



i = 0 2 + 0 = 2
i = 1 2 + 1 = 3
i = 2 2 + 2 = 4
i = 3 2 + 3 = 5

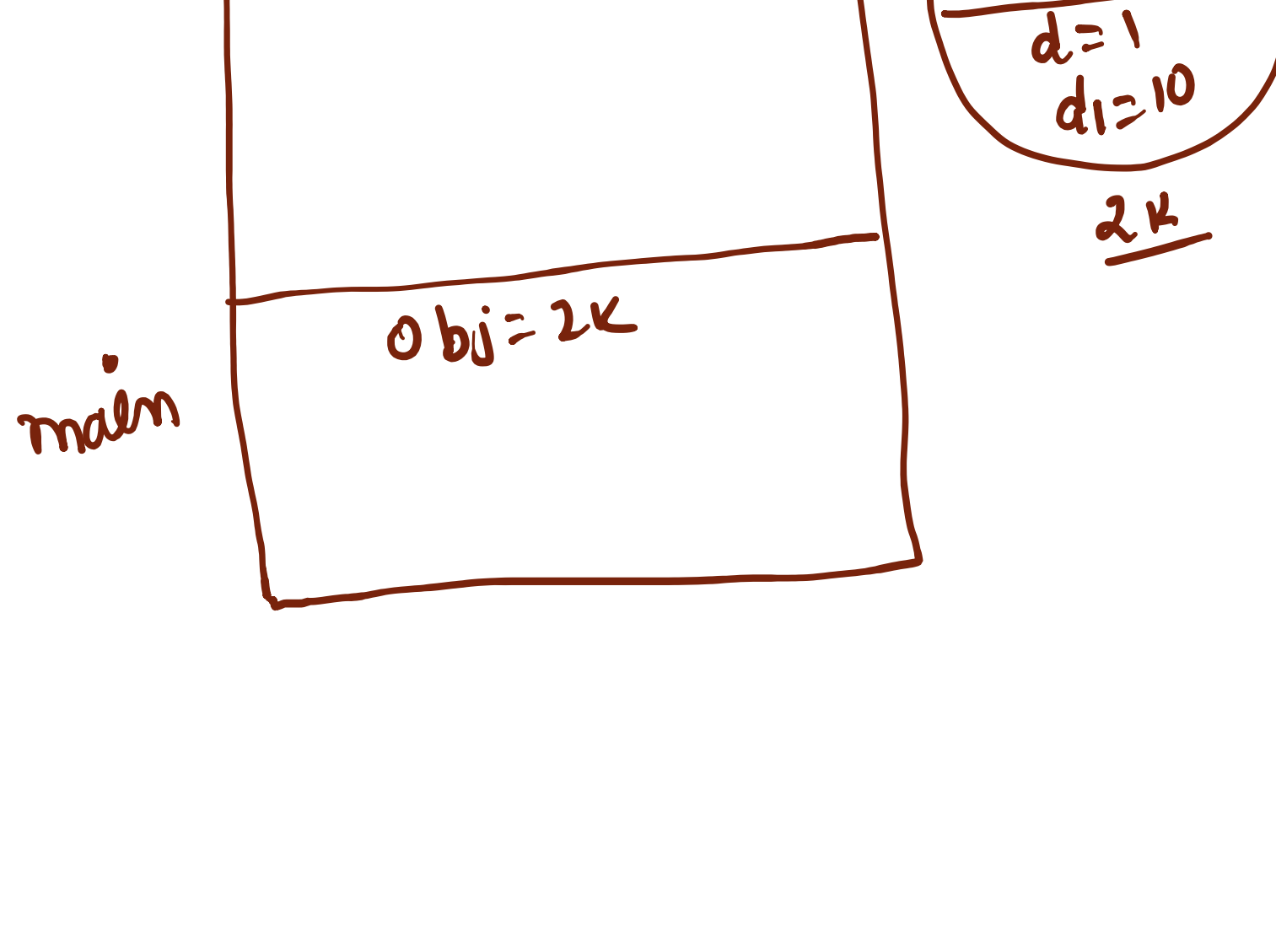


class Name var	new class Name()
P obj	new P()
P obj	new C()
C obj	new P()
C obj	new C()

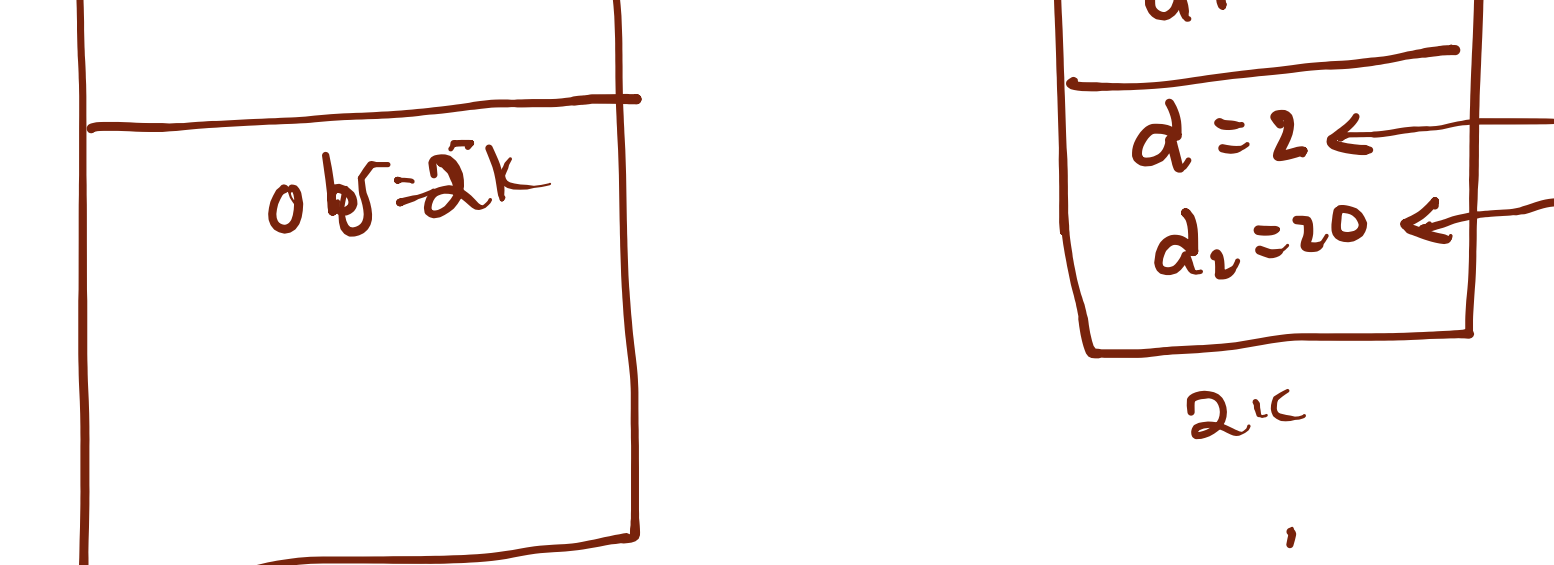
```
public static void main(String[] args) {  
    // case -1  
    P obj = new P();  
    System.out.println(obj.d);  
    System.out.println(obj.d1);  
    obj.fun();  
    obj.fun1();  
    // case -2  
    P obj = new C();  
    public class C extends P {  
        int d = 2;  
        int d2 = 20;  
        public void fun() {  
            System.out.println("Fun in C");  
        }  
        public void fun2() {  
            System.out.println("Fun2 in C");  
        }  
    }  
}
```

```
public class P {  
    int d = 1;  
    int d1 = 10;  
    public void fun() {  
        System.out.println("Fun in P");  
    }  
    public void fun1() {  
        System.out.println("Fun1 in P");  
    }  
}
```

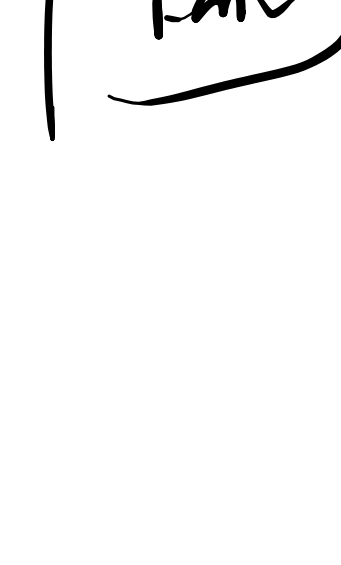
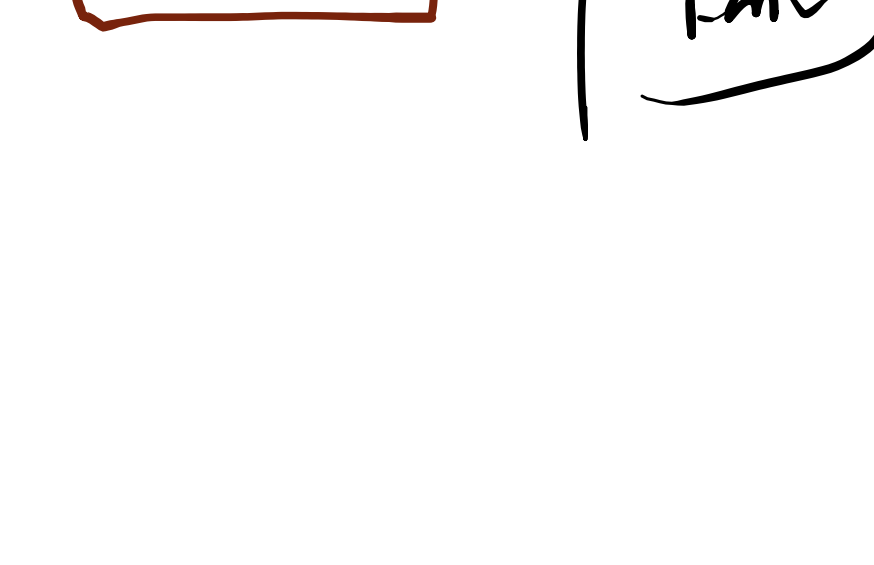
d1 fun fun1
d2 fun fun2



```
// case -2  
P obj = new C();  
System.out.println(obj.d);  
System.out.println(obj.d1);  
System.out.println(obj.d2);  
System.out.println(((C) obj).d);
```



```
public static void main(String[] args) {  
    // case -1  
    P obj = new P();  
    System.out.println(obj.d);  
    System.out.println(obj.d1);  
    obj.fun();  
    obj.fun1();  
    // case -2  
    C obj = new C();  
    System.out.println(obj.d);  
    System.out.println(obj.d1);  
    System.out.println(obj.d2);  
    obj.fun();  
    obj.fun1();  
    ((C) obj).fun2();  
}
```

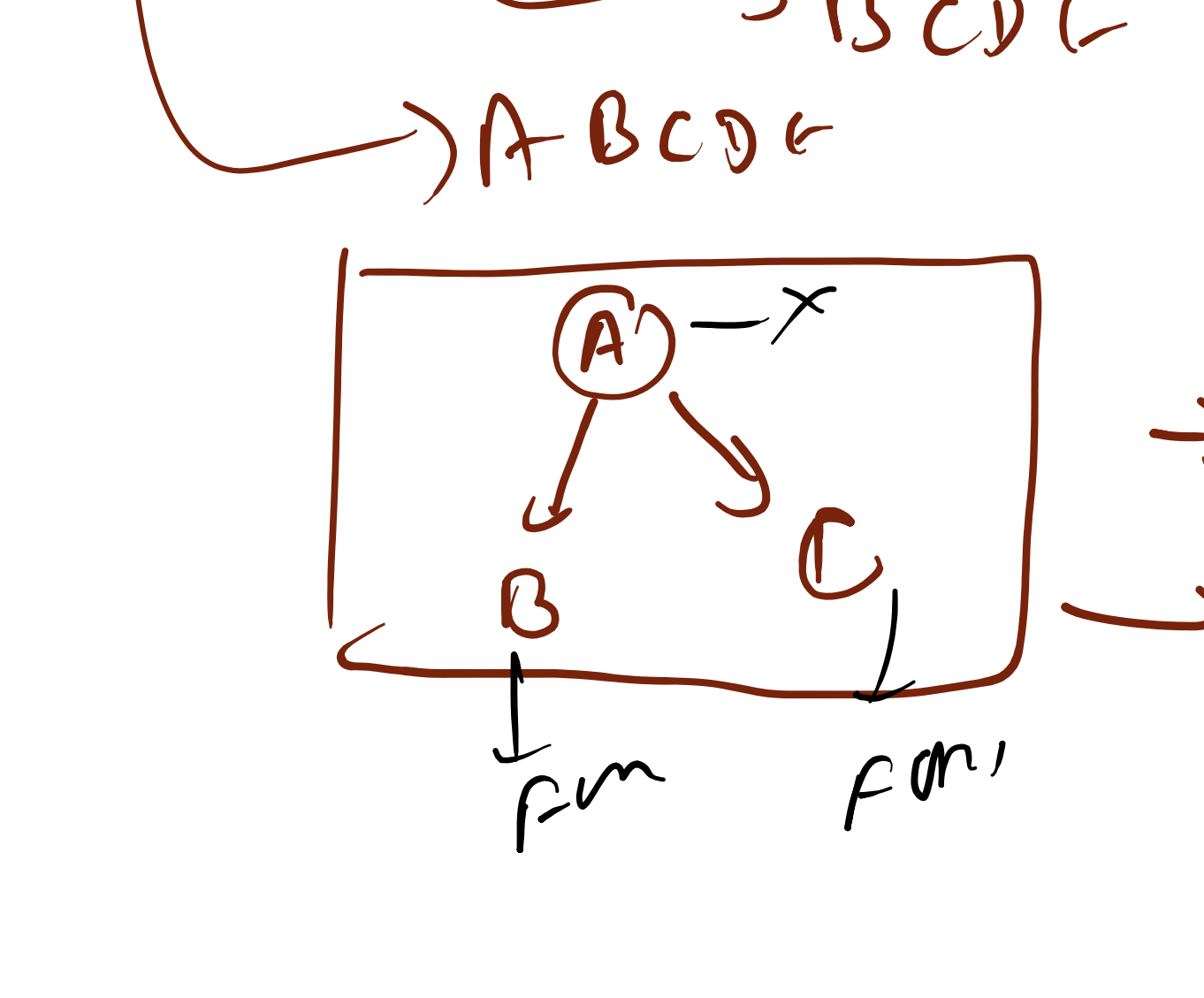


```
// case -2  
C obj = new P();  
System.out.println(obj.d);  
System.out.println(obj.d1);  
System.out.println(obj.d2);
```

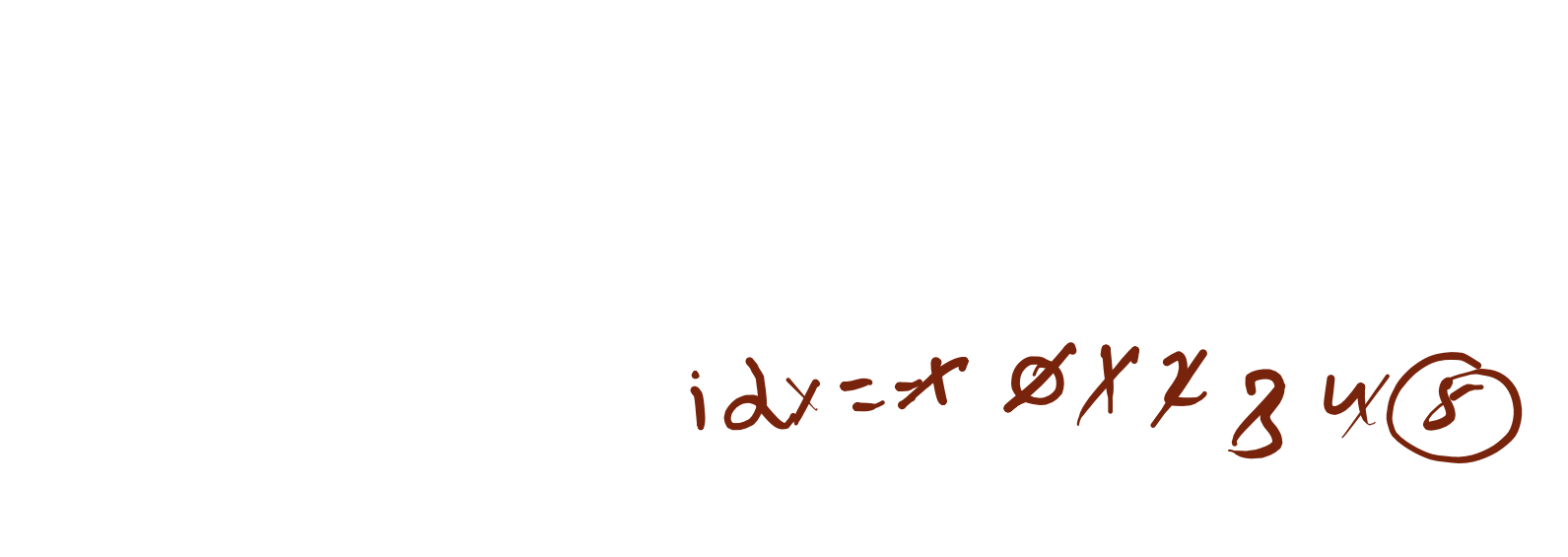


Multi-Down

Multiple



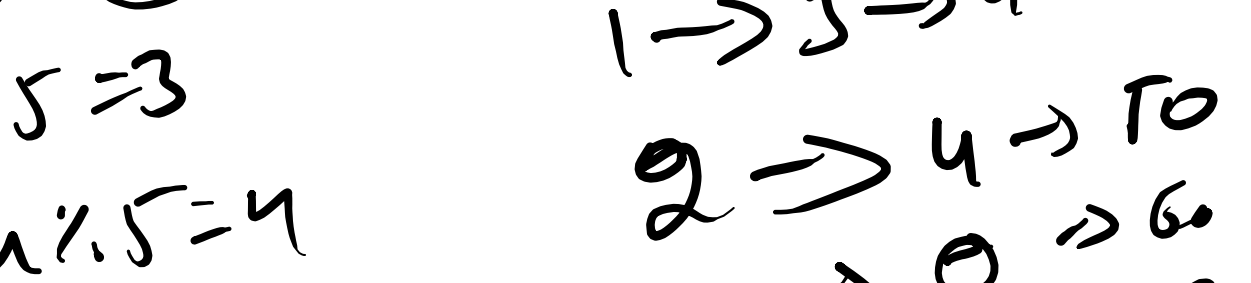
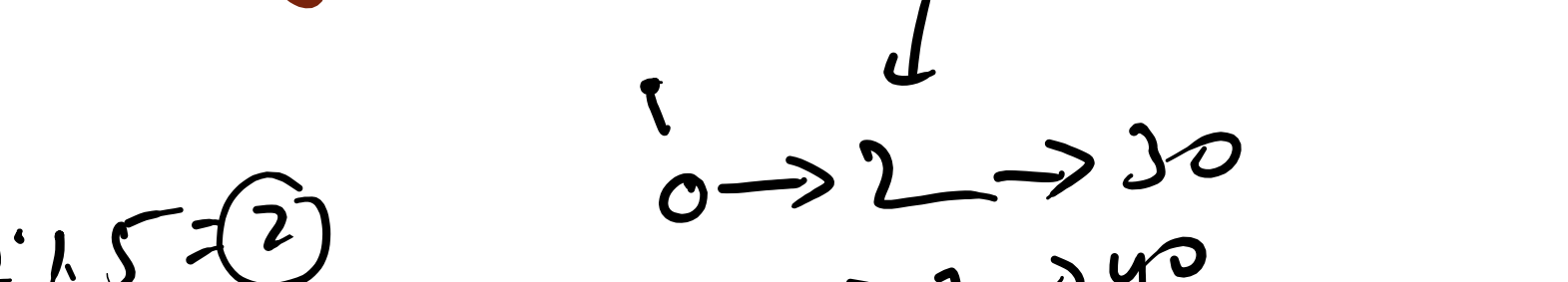
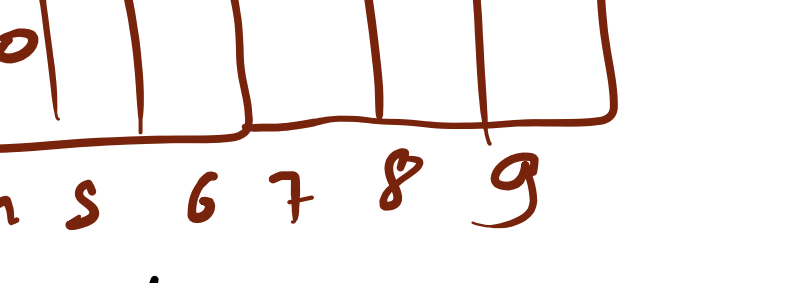
new A() obj.fun()
A obj = new A() obj.funC()



```
@Override  
public void push(int x) {  
    // ...  
}
```

idx = 1 0 1 2 3 4 5

```
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
    DynmaicStack ds = new DynmaicStack();  
    ds.push(10);  
    ds.push(20);  
    ds.push(30);  
    ds.push(40);  
    ds.push(50);  
    ds.push(60);  
}
```



idx = (i) % arr.length
newCid = arrCid

2+0 = 2 * 5 = 10
2+1 = 3 * 5 = 15
2+2 = 4 * 5 = 20
2+3 = 5 * 5 = 25
2+4 = 6 * 5 = 30

0 → 2 → 30
1 → 3 → 40
2 → 4 → 50
3 → 5 → 60
4 → 6 → 70